The development certificate is great for local work, but in production you'll need to configure a real, production certificate. I don't describe the process of obtaining a certificate in section 28.3, as that will vary by provider; instead, I show how to configure Kestrel to use a custom certificate you've acquired.

In section 28.4 I describe some of the approaches to enforcing HTTPS in your application. Unfortunately, web browsers still expect apps to be available over HTTP by default, so you typically need to expose your application on both HTTP and HTTPS ports. Nevertheless, there are things you can do to push clients toward the HTTPS endpoint, which are considered security best practices these days.

Before we look at HTTPS in ASP.NET Core specifically, we'll start by looking at HTTPS in general and why you should use it in all your applications.

28.1 Why do I need HTTPS?

In this section you'll learn about HTTPS: what it is, and why you need to be aware of it for all your production applications. We're not going to go into details about the protocol or how certificates work at this point, instead focusing on why you need to use HTTPS. You'll see two approaches to adding HTTPS to your application: supporting HTTPS directly in your application and using SSL/TLS-offloading with a reverse proxy.

So far in this book, I've shown how the user's browser sends a request across the internet to your app using the HTTP protocol. We haven't looked too much into the details of that protocol other than to establish that it uses *verbs* to describe the type of request (such as `GET` and `POST`), that it